Manual de Configuracion - Entrega a Cliente

1) Prerrequisitos

- Cuenta Google para el negocio.
- Acceso a Google Cloud Console.
- Node.js 20+.
- Cuenta de despliegue (ej. Vercel).

2) Configuracion en Google Cloud (obligatoria)

2.1 Crear proyecto

1. Crear proyecto nuevo en Google Cloud.
2. Guardar el Project ID.

2.2 Activar API

1. Activar `Google Calendar API`.

2.3 Crear Service Account (bot)

1. Ir a IAM y Admin -> Service Accounts.
2. Crear cuenta, por ejemplo `reserva-bot`.
3. Crear clave JSON y descargarla.
4. Del JSON tomar:

- `client_email` -> `GOOGLE_CLIENT_EMAIL`
- `private_key` -> `GOOGLE_PRIVATE_KEY`

2.4 Compartir calendario con el bot

1. Abrir Google Calendar del negocio.
2. Ir a configuracion del calendario objetivo.
3. Compartir con el `client_email` del bot.
4. Permiso recomendado: `Make changes to events`.

2.5 Obtener Calendar ID real

1. En configuracion del calendario -> Integrar calendario.
2. Copiar `Calendar ID`.
3. Guardar en `NEXT_PUBLIC_CALENDAR_ID`.

2.6 Configurar OAuth 2.0 para login admin

1. Configurar OAuth consent screen.
2. Crear `OAuth Client ID` tipo `Web application`.
3. Registrar:

- Authorized JavaScript origins:
  - `http://localhost:3000`
  - `https://tu-dominio.com`
- Authorized redirect URIs:

- `http://localhost:3000/api/auth/callback/google`
- `https://tu-dominio.com/api/auth/callback/google`

4. Copiar:

- `Client ID` -> `GOOGLE_CLIENT_ID`
- `Client Secret` -> `GOOGLE_CLIENT_SECRET`

Nota:

- Si la app esta en modo testing, agregar los correos admin en Test Users.

2.7 Scopes que usa el proyecto

Scopes configurados en el login:

- `openid`
- `email`
- `profile`
- `https://www.googleapis.com/auth/calendar.events`
- `https://www.googleapis.com/auth/calendar.readonly`

3) Configuracion de Gmail para envio de correos

1. Activar 2FA en la cuenta Gmail emisora.
2. Crear App Password de 16 caracteres.
3. Configurar:

- `GMAIL_USER`
- `GMAIL_APP_PASSWORD`

4) Variables de entorno a completar

Crear `.env.local` en raiz del proyecto:

```
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
ADMIN_EMAILS=
GOOGLE_CLIENT_EMAIL=
GOOGLE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
NEXT_PUBLIC_CALENDAR_ID=
GMAIL_USER=
GMAIL_APP_PASSWORD=
CRON_SECRET=
```

5) Ejecucion local

1. Instalar dependencias:

```
npm install
```

2. Levantar entorno local:

```
npm run dev
```

3. Abrir:

- `http://localhost:3000`
- `http://localhost:3000/admin/reservas`

6) Deploy (resumen)

1. Subir repo privado.
2. Conectar proyecto en Vercel.
3. Cargar variables de entorno en `Production`.
4. Ajustar `NEXTAUTH_URL` al dominio real.
5. Deploy.

7) Checklist de validacion final

1. Login admin Google funciona.
2. El correo admin esta incluido en `ADMIN_EMAILS`.
3. `free-busy` responde sin error 500.
4. Reserva pendiente se crea.
5. Admin puede aprobar reserva.
6. Correo al cliente llega correctamente.
7. Cron cleanup protegido por `CRON_SECRET`.

8) Troubleshooting rapido

Error `invalid_client` en login Google

- Revisar `GOOGLE_CLIENT_ID` y `GOOGLE_CLIENT_SECRET`.
- Confirmar redirect URI exacta.

Error `invalid_grant: account not found` en Calendar

- Revisar coherencia entre `GOOGLE_CLIENT_EMAIL` y `GOOGLE_PRIVATE_KEY`.
- Confirmar que el bot tiene acceso al calendario.

Error 500 en `/api/calendar/free-busy`

- Revisar `NEXT_PUBLIC_CALENDAR_ID`.
- Confirmar Calendar API activa.